

```
In [2]: 1 import pandas as pd
2 import seaborn as sns
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 from sklearn.model_selection import (train_test_split, GridSearchCV, cross_val_score)
7
8 from sklearn.neighbors import KNeighborsClassifier
9
10 from sklearn.preprocessing import StandardScaler
11
12 from sklearn.metrics import (accuracy_score, classification_report, confusion_matrix)
13
14 import warnings
15 warnings.filterwarnings("ignore")
16
17 sns.set_theme(style="whitegrid")
```

```
In [3]: 1 #Step 1:Load dataset
2 df = pd.read_csv("diabetes.csv")
```

```
In [4]: 1 df.head()
2 df.tail()
3 df.shape
4 df.isnull()
5 df.columns
6 df.describe()
7 df.info()
8 df.isnull().sum()
9 df.duplicated().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 308 entries, 0 to 307
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Pregnancies            308 non-null   int64
1   Glucose                308 non-null   int64
2   BloodPressure          308 non-null   int64
3   SkinThickness          308 non-null   int64
4   Insulin                308 non-null   int64
5   BMI                    308 non-null   float64
6   DiabetesPedigreeFunction 308 non-null   float64
7   Age                    308 non-null   int64
8   Outcome                308 non-null   int64
dtypes: float64(2), int64(7)
memory usage: 21.8 KB
```

Out[4]: 0

```
In [5]: 1 zero_columns = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "  
2 zero_counts = (df[zero_columns] == 0).sum()  
3 print("Zero counts:", zero_counts)
```

```
Zero counts: Glucose      1  
BloodPressure    14  
SkinThickness    93  
Insulin          152  
BMI              4  
dtype: int64
```

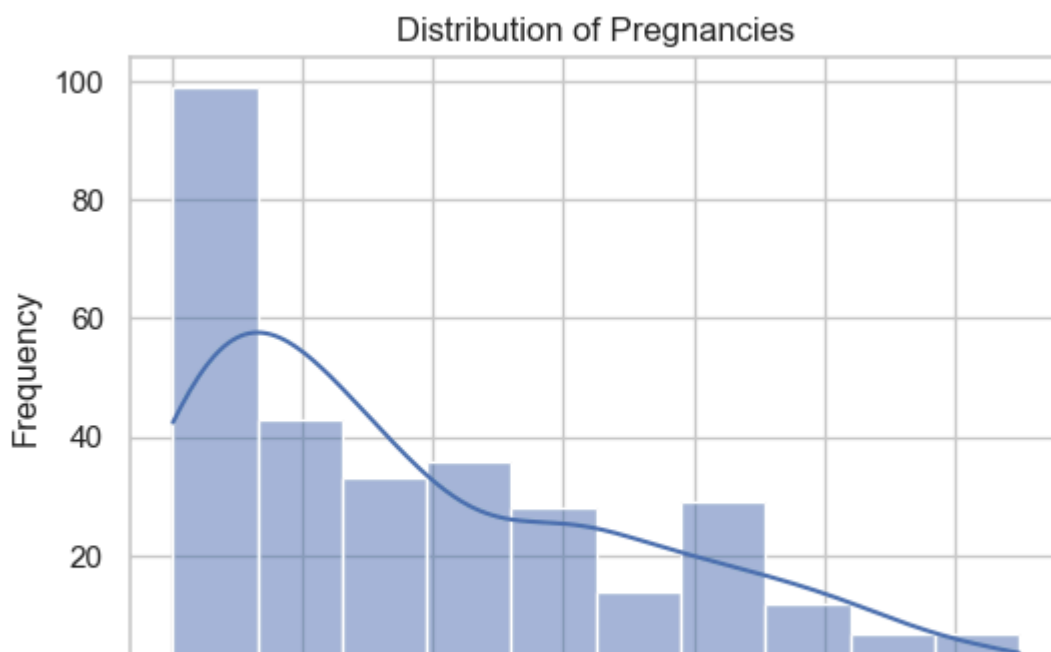
```
In [6]: 1 df_clean = df.copy()  
2  
3 df_clean[zero_columns] = df_clean[zero_columns].replace(0, np.nan)  
4  
5 print("Missing values after replacing invalid zeros:")  
6 print(df_clean.isnull().sum())
```

```
Missing values after replacing invalid zeros:  
Pregnancies      0  
Glucose          1  
BloodPressure    14  
SkinThickness    93  
Insulin          152  
BMI              4  
DiabetesPedigreeFunction  0  
Age              0  
Outcome          0  
dtype: int64
```

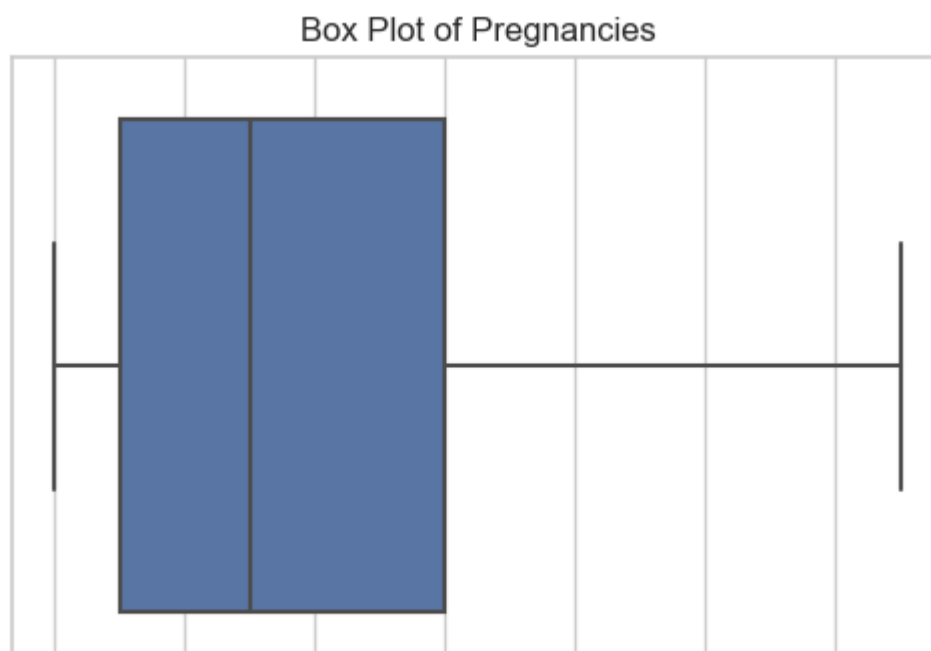
```
In [7]: 1 for col in zero_columns:  
2     df_clean[col] = df_clean[col].fillna(df_clean[col].median())  
3 print("Missing values after median imputation:")  
4 print(df_clean.isnull().sum())
```

```
Missing values after median imputation:  
Pregnancies      0  
Glucose          0  
BloodPressure    0  
SkinThickness    0  
Insulin          0  
BMI              0  
DiabetesPedigreeFunction  0  
Age              0  
Outcome          0  
dtype: int64
```

```
In [8]: 1 numeric_columns = df_clean.select_dtypes(include=np.number).columns
2
3 for col in numeric_columns:
4     plt.figure(figsize=(6, 4))
5     sns.histplot(df_clean[col], kde=True)
6     plt.title("Distribution of " + col)
7     plt.xlabel(col)
8     plt.ylabel("Frequency")
9     plt.show()
```



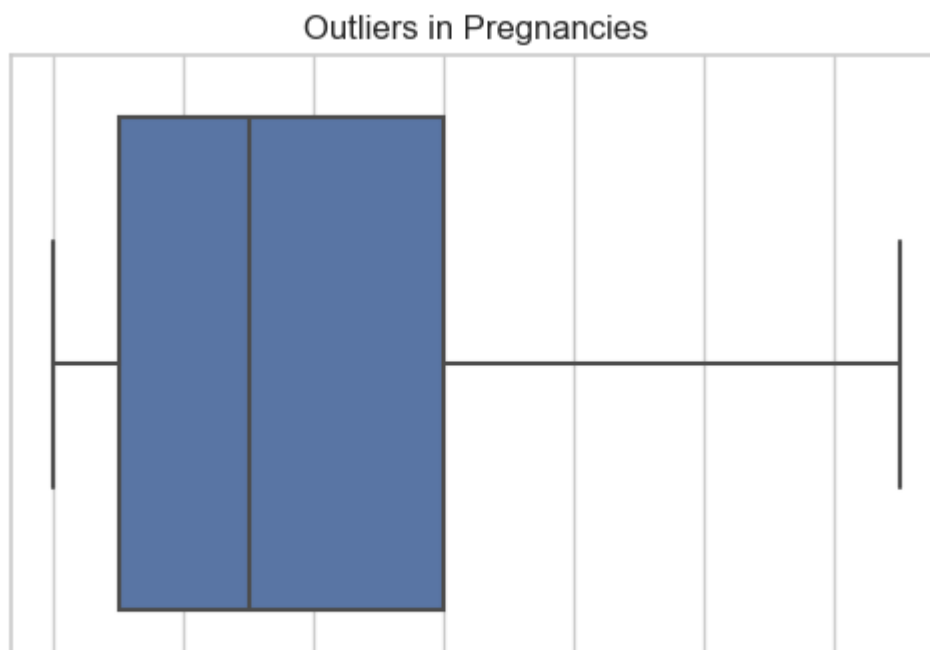
```
In [9]: 1 numeric_columns = df_clean.select_dtypes(include=np.number).columns
2
3 for col in numeric_columns:
4     plt.figure(figsize=(6, 4))
5     sns.boxplot(x=df_clean[col])
6     plt.title("Box Plot of " + col)
7     plt.xlabel(col)
8     plt.show()
```



```

In [10]: 1 #Outliers
2
3 numeric_columns = df_clean.select_dtypes(include=np.number).columns
4
5 for col in numeric_columns:
6     plt.figure(figsize=(6,4))
7     sns.boxplot(x=df_clean[col])
8     plt.title("Outliers in " + col)
9     plt.show()

```



```

In [11]: 1 #IQR
2 #IQR
3
4 Q1 = df_clean[numeric_columns].quantile(0.25)
5 Q3 = df_clean[numeric_columns].quantile(0.75)
6
7 IQR = Q3 - Q1
8
9 lower_bound = Q1 - 1.5 * IQR
10 upper_bound = Q3 + 1.5 * IQR
11
12 outliers = ((df_clean[numeric_columns] < lower_bound) |
13             (df_clean[numeric_columns] > upper_bound))
14
15 print("Number of outliers in each column:")
16 print(outliers.sum())

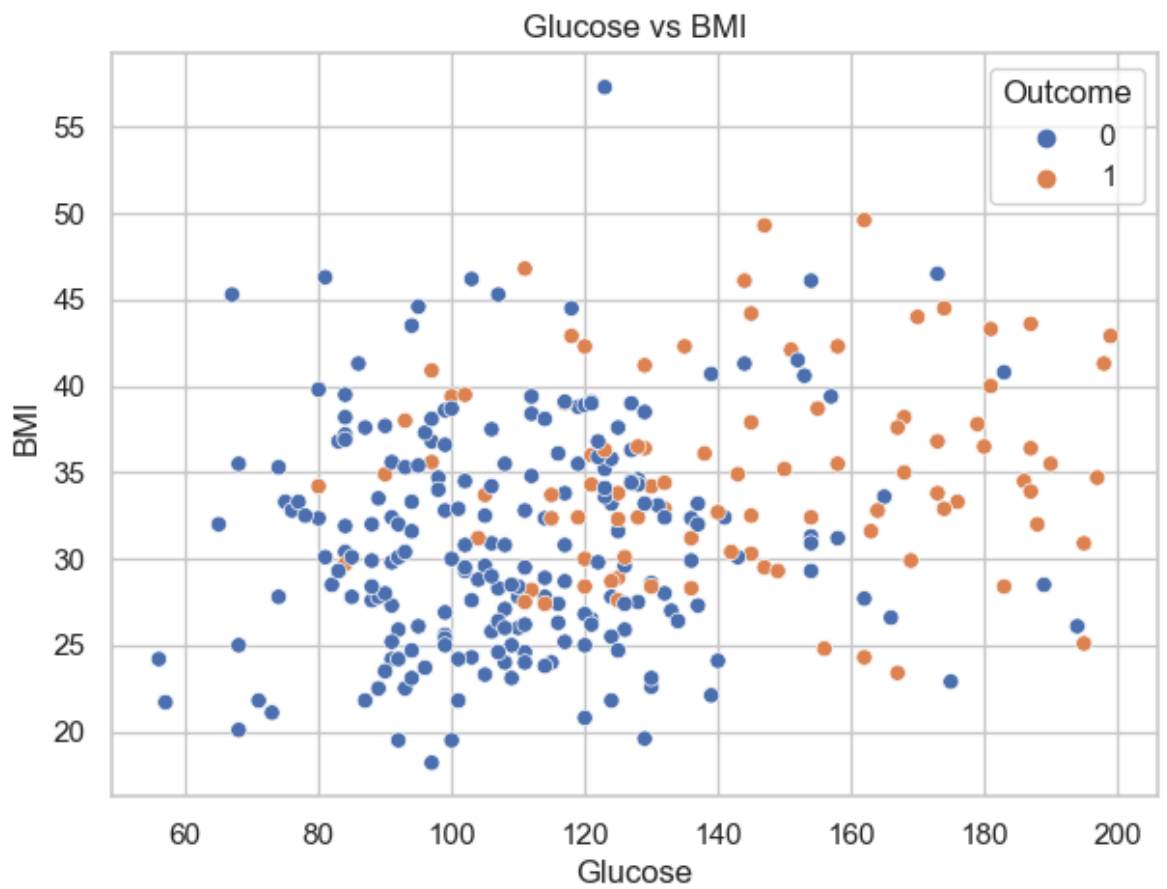
```

```

Number of outliers in each column:
Pregnancies          0
Glucose              6
BloodPressure        7
SkinThickness       38
Insulin             145
BMI                  1
DiabetesPedigreeFunction  12
Age                  1
Outcome              0
dtype: int64

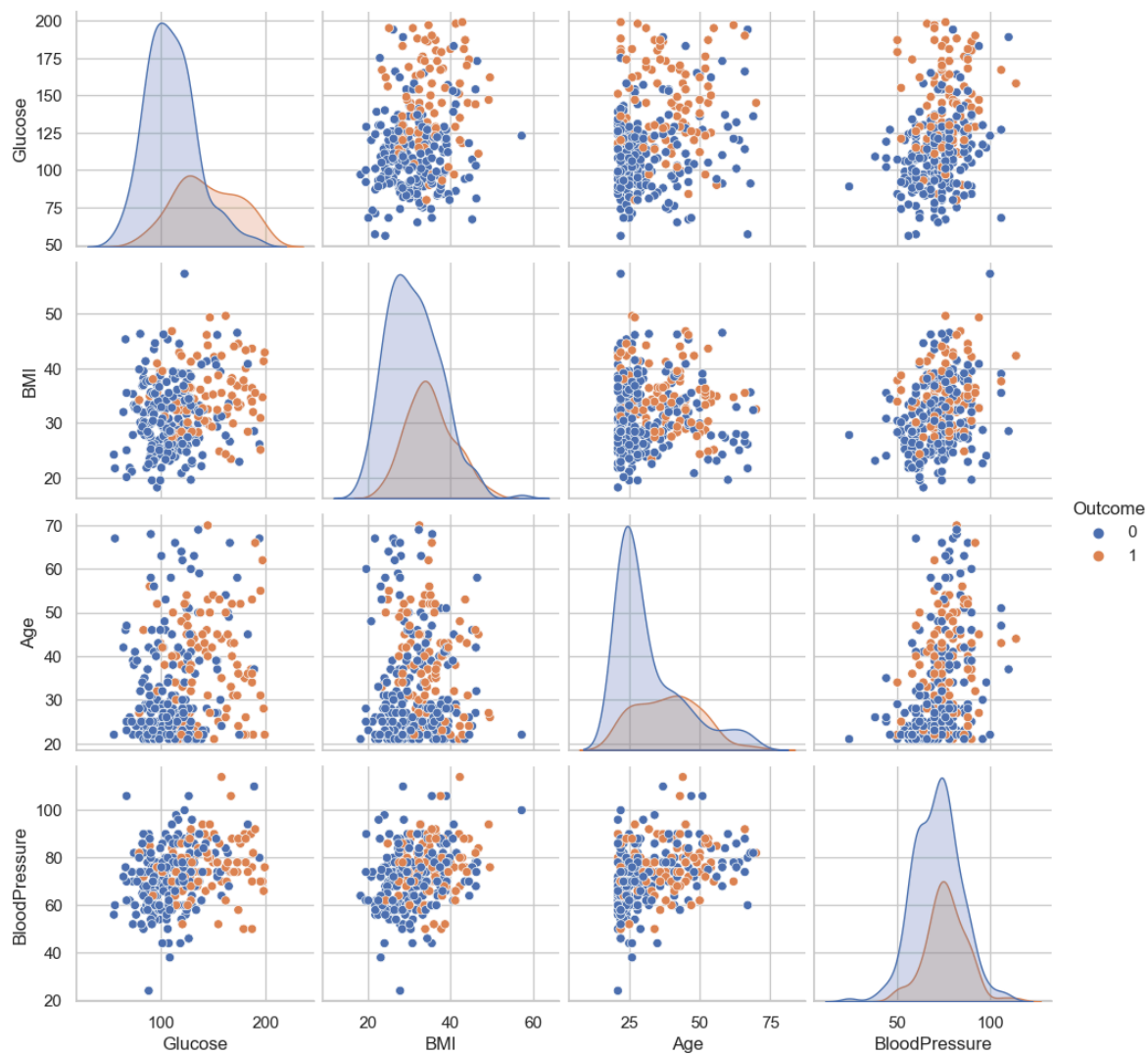
```

```
In [12]: 1 #Bi-variate analysis
2 #Bi-variate analysis
3
4 plt.figure(figsize=(7,5))
5 sns.scatterplot(data=df_clean, x="Glucose", y="BMI", hue="Outcome")
6 plt.title("Glucose vs BMI")
7 plt.show()
```



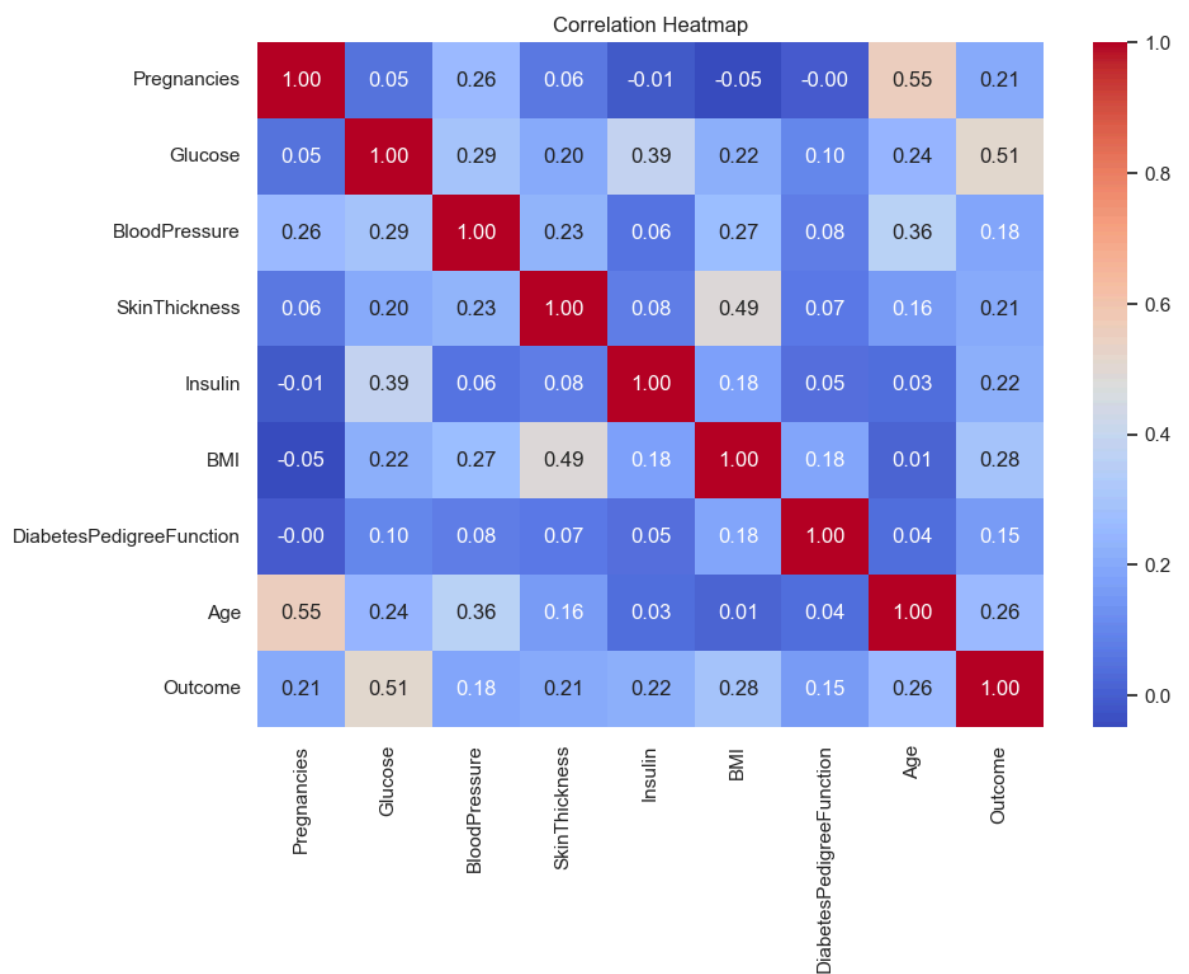
In [13]:

```
1 #Multivariate analysis
2 #Multivariate analysis
3
4 sns.pairplot(
5     df_clean[["Glucose", "BMI", "Age", "BloodPressure", "Outcome"]],
6     hue="Outcome"
7 )
8
9 plt.show()
```



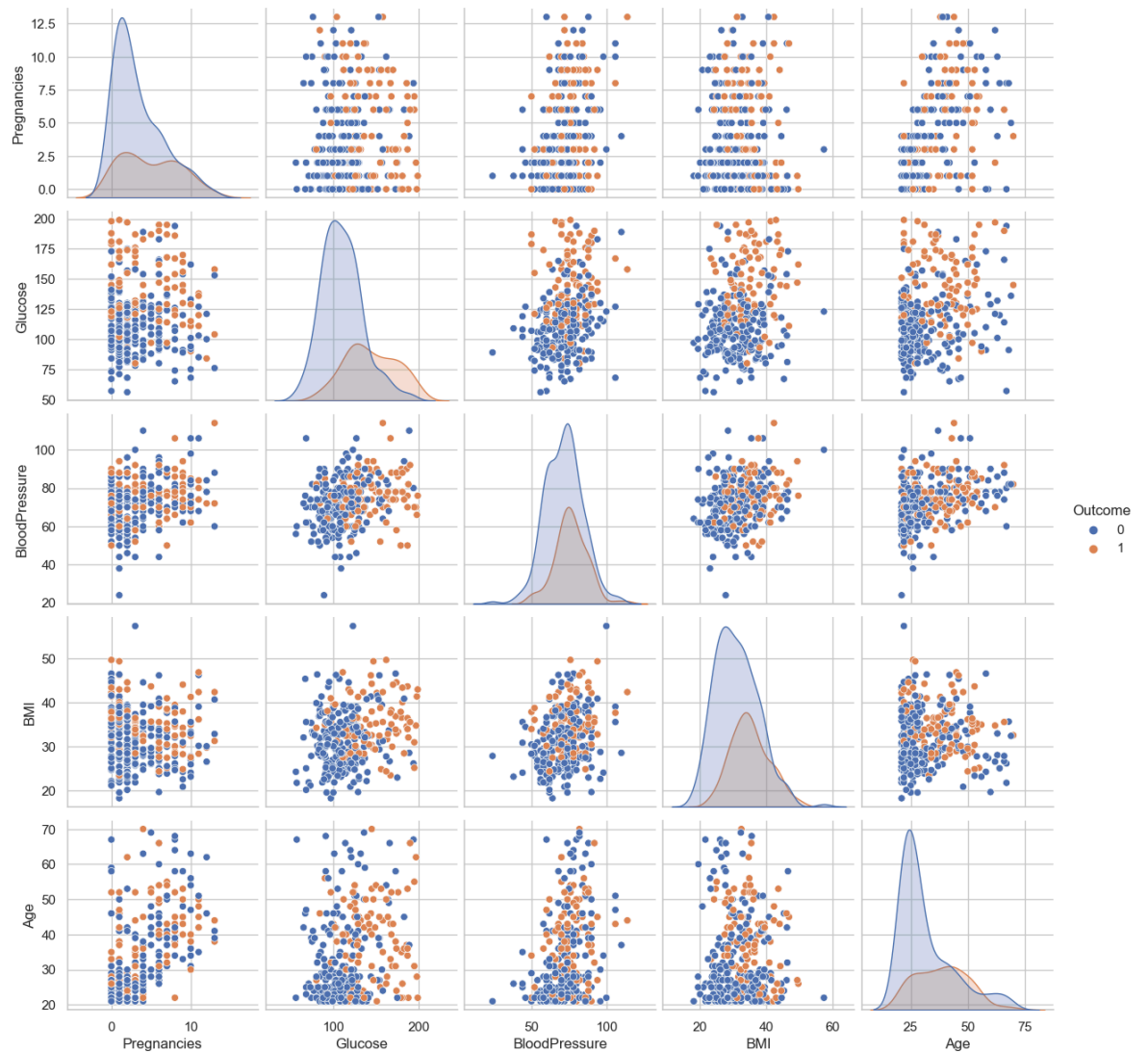
In [14]:

```
1 #Heatmap
2 #Heatmap
3
4 plt.figure(figsize=(10,7))
5
6 correlation = df_clean.corr()
7
8 sns.heatmap(
9     correlation,
10     annot=True,
11     cmap="coolwarm",
12     fmt=".2f"
13 )
14
15 plt.title("Correlation Heatmap")
16 plt.show()
```



In [15]:

```
1 #Pair plot
2 #Pair plot
3
4 sns.pairplot(
5     df_clean,
6     vars=["Pregnancies", "Glucose", "BloodPressure", "BMI", "Age"],
7     hue="Outcome"
8 )
9
10 plt.show()
```



In [16]:

```
1  from sklearn.metrics import (
2      accuracy_score,
3      classification_report,
4      confusion_matrix,
5      precision_score,
6      recall_score,
7      f1_score
8  )
9
10 # Apply KNN
11
12 X = df_clean.drop("Outcome", axis=1)
13 y = df_clean["Outcome"]
14
15 # Split dataset
16 X_train, X_test, y_train, y_test = train_test_split(
17     X, y,
18     test_size=0.2,
19     random_state=42,
20     stratify=y
21 )
22
23 print(y_train.value_counts(normalize=True))
24 print(y_test.value_counts(normalize=True))
25
26 # Feature scaling
27 scaler = StandardScaler()
28
29 X_train = scaler.fit_transform(X_train)
30 X_test = scaler.transform(X_test)
31
32 # Create KNN model
33 knn = KNeighborsClassifier(n_neighbors=5)
34
35 # Train model
36 knn.fit(X_train, y_train)
37
38 # Prediction
39 y_pred = knn.predict(X_test)
40 y_prob = knn.predict_proba(X_test)[: , 1]
41
42 # Accuracy
43 accuracy = accuracy_score(y_test, y_pred)
44 print("KNN Accuracy:", accuracy)
45
46 # Precision
47 precision = precision_score(y_test, y_pred)
48 print("KNN Precision:", precision)
49
50 # Recall
51 recall = recall_score(y_test, y_pred)
52 print("KNN Recall:", recall)
53
54 # F1 Score
55 f1 = f1_score(y_test, y_pred)
56 print("KNN F1 Score:", f1)
57
58 # Classification report
59 print("\nClassification Report:")
60 print(classification_report(y_test, y_pred))
61
```

```

62 # Confusion matrix
63 cm = confusion_matrix(y_test, y_pred)
64
65 print("\nConfusion Matrix:")
66 print(cm)

```

Outcome

0 0.699187

1 0.300813

Name: proportion, dtype: float64

Outcome

0 0.693548

1 0.306452

Name: proportion, dtype: float64

KNN Accuracy: 0.7096774193548387

KNN Precision: 0.5238095238095238

KNN Recall: 0.5789473684210527

KNN F1 Score: 0.5500000000000002

Classification Report:

	precision	recall	f1-score	support
0	0.80	0.77	0.79	43
1	0.52	0.58	0.55	19
accuracy			0.71	62
macro avg	0.66	0.67	0.67	62
weighted avg	0.72	0.71	0.71	62

Confusion Matrix:

[[33 10]

[8 11]]

```

In [17]: 1 # Confusion matrix
2 cm = confusion_matrix(y_test, y_pred)
3
4 print("\nConfusion Matrix:")
5 print(cm)
6
7 # Plot Confusion Matrix
8 plt.figure(figsize=(6, 5))
9
10 sns.heatmap(
11     cm,
12     annot=True,
13     fmt="d",
14     cmap="Blues",
15     xticklabels=["No Diabetes", "Diabetes"],
16     yticklabels=["No Diabetes", "Diabetes"]
17 )
18
19 plt.xlabel("Predicted")
20 plt.ylabel("Actual")
21 plt.title("KNN Confusion Matrix")
22 plt.show()

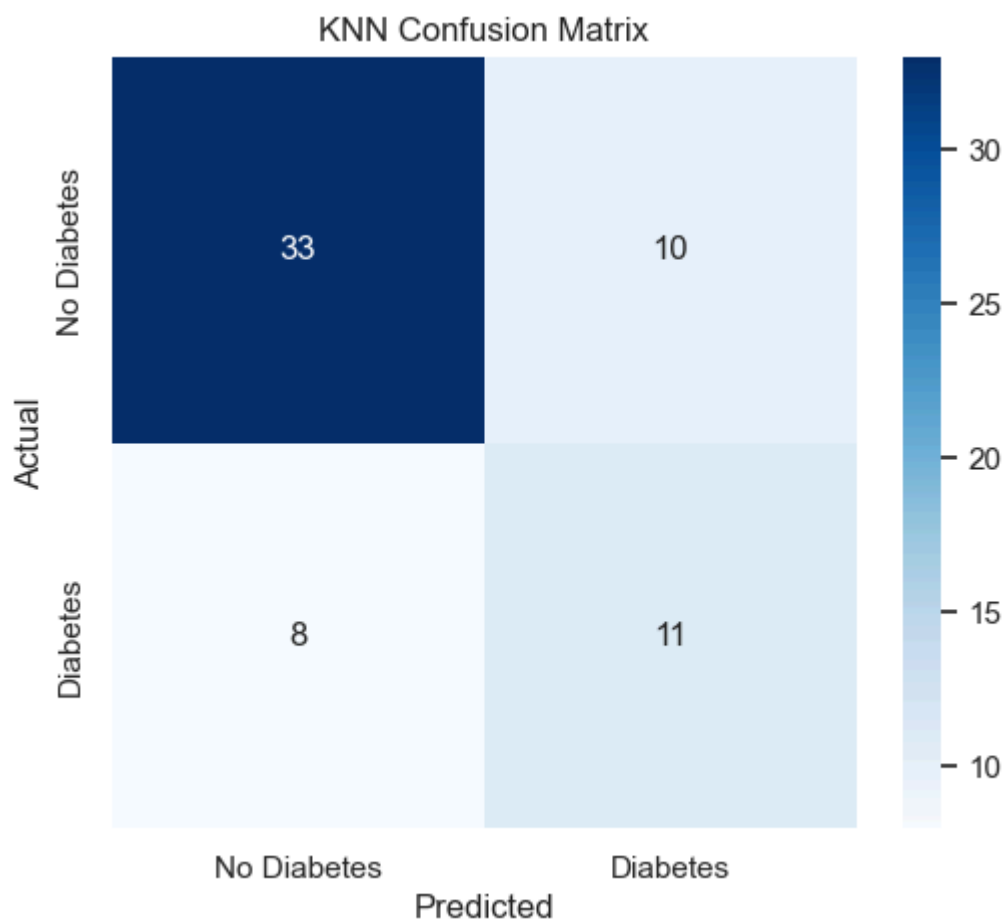
```

Confusion Matrix:

```

[[33 10]
 [ 8 11]]

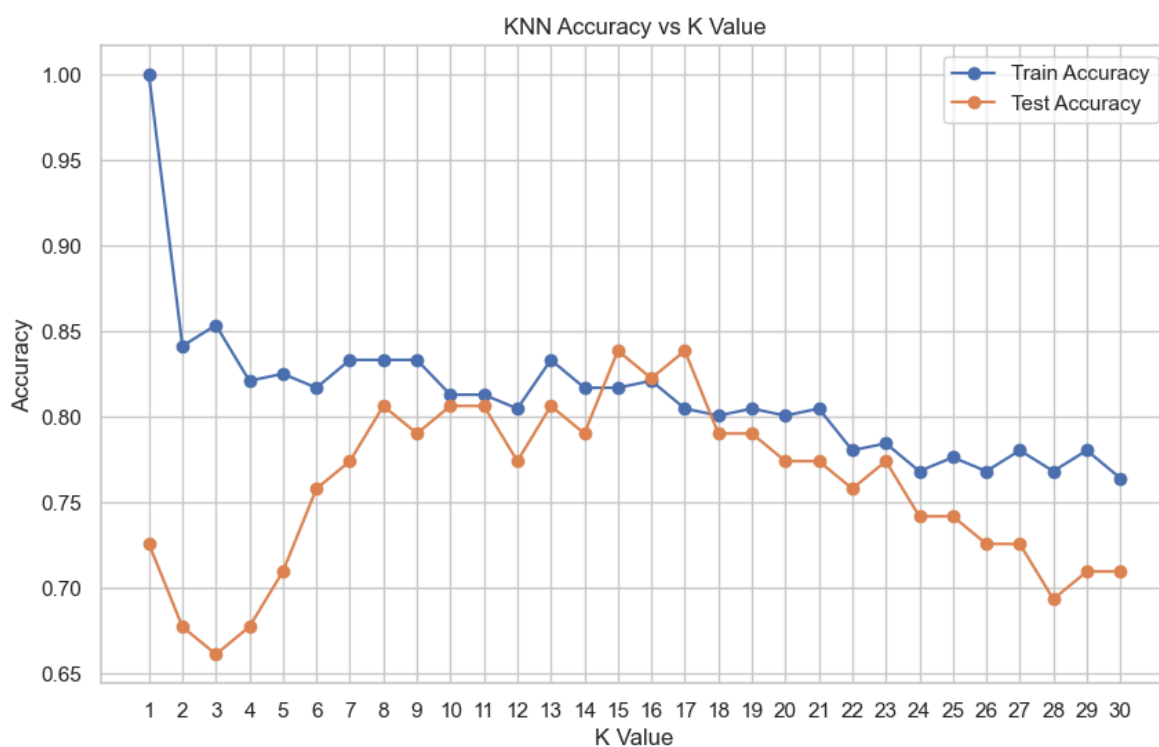
```



```

In [22]: 1 k_values = range(1, 31)
          2
          3 train_accuracy = []
          4 test_accuracy = []
          5
          6 for k in k_values:
          7     model = KNeighborsClassifier(n_neighbors=k)
          8
          9     model.fit(X_train, y_train)
         10
         11     train_accuracy.append(model.score(X_train, y_train))
         12     test_accuracy.append(model.score(X_test, y_test))
         13
         14
         15 plt.figure(figsize=(10, 6))
         16
         17 plt.plot(k_values, train_accuracy, marker='o', label="Train Accuracy")
         18 plt.plot(k_values, test_accuracy, marker='o', label="Test Accuracy")
         19
         20 plt.xlabel("K Value")
         21 plt.ylabel("Accuracy")
         22 plt.title("KNN Accuracy vs K Value")
         23
         24 plt.xticks(k_values)
         25 plt.legend()
         26 plt.grid(True)
         27
         28 plt.show()

```



```
In [23]: 1 best_k = k_values[np.argmax(test_accuracy)]
2 best_accuracy = max(test_accuracy)
3
4 print("Best K:", best_k)
5 print("Best Test Accuracy:", best_accuracy)
```

Best K: 15

Best Test Accuracy: 0.8387096774193549

```
In [24]: 1 cv = StratifiedKFold(n_splits=5,shuffle=True,random_state=42)
2 k_values = range(1, 31)
3 cv_scores = []
4
5 for k in k_values:
6     model = KNeighborsClassifier(n_neighbors=k)
7
8     scores = cross_val_score(
9         model,
10        X_train,
11        y_train,
12        cv=5,
13        scoring="accuracy"
14    )
15
16    cv_scores.append(scores.mean())
17
18 # Find best K using argmax
19 best_index = np.argmax(cv_scores)
20 best_k = list(k_values)[best_index]
21 best_cv_score = cv_scores[best_index]
22
23 print("Best K:", best_k)
24 print("Best CV Score:", best_cv_score)
```

Best K: 11

Best CV Score: 0.7848163265306123

```
In [25]: 1 param_grid = {
2     "n_neighbors": range(1, 31),
3     "weights": ["uniform", "distance"],
4     "metric": ["euclidean", "manhattan", "minkowski"],
5     "p": [1,2]
6 }
```

```
In [26]: 1 grid_search = GridSearchCV(
2         estimator=KNeighborsClassifier(),
3         param_grid=param_grid,
4         scoring="accuracy",
5         n_jobs=-1,
6         cv=5
7     )
8     grid_search.fit(X_train, y_train)
```

```
Out[26]: ▸ GridSearchCV
          ▸ estimator: KNeighborsClassifier
              ▾ KNeighborsClassifier
                  KNeighborsClassifier()
```

```
In [27]: 1 # Best parameters
2         print("Best Parameters:", grid_search.best_params_)
3
4         # Best CV score
5         print("Best CV Score:", grid_search.best_score_)
```

Best Parameters: {'metric': 'euclidean', 'n_neighbors': 11, 'p': 1, 'weights': 'uniform'}

Best CV Score: 0.7848163265306123

```
In [ ]: 1
```